L’analyse

Cette fiche présente :

1 **Fondements et enjeux de l’analyse informatique**
2 **Les méthodes d’analyse : cartésienne, systémique et objet**
3 **Modélisation et stratégies : l’approche Merise et ses niveaux d’abstraction**

1. Fondements et enjeux de l’analyse informatique

1.1. Qu’est-ce que l’analyse informatique ?

L’**analyse informatique** est la phase cruciale du développement d’un système dont l’objectif principal est de **formaliser un modèle du domaine du problème**, indépendant de toute considération d’implémentation. Elle se concentre sur **ce qui doit être fait**, et non sur **comment le faire**. En d’autres termes, elle vise à :

- **Comprendre et formaliser** les besoins d’informatisation exprimés par les utilisateurs.
- **Traduire ces besoins** en une forme compréhensible pour les développeurs, afin qu’ils puissent les transformer en programmes et en structures de données.
- **Produire des spécifications** (ou « spécifications fonctionnelles »), qui serviront de référence tout au long du projet.

Dans une **approche orientée objet**, l’analyse se décompose en deux étapes complémentaires :

- **L’analyse fonctionnelle** (ou « spécifications fonctionnelles générales ») : basée sur les **cas d’utilisation**, elle recense l’ensemble des fonctionnalités nécessaires au bon fonctionnement de l’application.
- **L’analyse détaillée** (ou « spécifications fonctionnelles détaillées ») : plus orientée objet, elle rassemble tous les concepts issus de l’analyse fonctionnelle pour produire un modèle complet, incluant les **contraintes**, les **règles de gestion** et les **services** attendus.

1.2. Le processus de construction d'une application : une suite de traductions

La construction d'une application informatique est un processus **itératif et cumulatif**, qui consiste en une **série de traductions** :

1. **Étude de l'existant** : analyser le système physique (procédures manuelles ou informatiques en place) pour en extraire une **définition abstraite**, mettant en lumière la **logique des processus métier** et les **informations** qu'ils manipulent.
2. **Expression des besoins** : avec les futurs utilisateurs, formaliser les **nouvelles fonctionnalités** attendues, ainsi que les **corrections des dysfonctionnements** identifiés dans l'existant.
3. **Construction de la nouvelle application** : utiliser cette définition du système comme base pour développer la solution informatique.

Limites de l'approche historique :

Cette méthode fonctionne bien si la nouvelle application conserve **au moins 50 % des fonctionnalités de l'ancienne**. Cependant, dans la plupart des cas, l'application existante est **obsolète**, mal documentée, et sa conception a souffert de maintenances successives. Il est alors préférable de **partir des nouveaux besoins** et de construire directement la nouvelle application, en confrontant ensuite les procédures résultantes de l'analyse aux procédures existantes pour en apprécier la faisabilité.

1.3. Les objectifs de la phase d'analyse

La phase d'analyse poursuit plusieurs objectifs fondamentaux :

- **Valider l'exhaustivité et la pertinence** des besoins exprimés par la maîtrise d'ouvrage (MOA).
- **Retranscrire ces besoins** dans une forme compréhensible par toutes les parties prenantes, pour s'assurer d'une **bonne compréhension mutuelle** sur le périmètre, les objectifs, les concepts métier et les règles de gestion.
- **Construire une architecture logicielle cohérente** avec les besoins exprimés.
- **Atténuer la complexité** du futur système en proposant une **vision formalisée**, déconnectée des contraintes techniques d'implémentation.
- **Documenter le futur système** pour faciliter sa validation et préparer les **évolutions futures**. La documentation n'est pas destinée au présent, mais à la **maintenance** du système.
- **Assurer une traçabilité** entre l'expression des besoins et le logiciel final, pour mesurer l'impact des changements (notamment technologiques) sur le métier.

2. Les méthodes d'analyse : cartésienne, systémique et objet

2.1. L'analyse cartésienne (1ère génération) : découper pour mieux régner

Principe :

Cette approche, popularisée dans les années 1970, consiste à **découper une fonction en sous-fonctions**, de manière hiérarchique, jusqu'à atteindre un niveau suffisamment fin pour permettre un codage simple. Elle s'appuie sur des méthodes comme **SADT**, **Jackson Structured Programming** ou **Yourdon**.

Points forts :

- **Simplicité** : la démarche est intuitive et s'appuie sur le bon sens.
- **Facilité de capture des besoins** : adaptée pour formaliser des processus métiers clairs.
- **Niveaux d'abstraction multiples** : permet une analyse progressive.

Points faibles :

- **Négligence de la cohérence des données** : risque de redondance (une même donnée dans plusieurs modules).
- **Absence de règles de décomposition explicites** : chaque développeur peut décomposer différemment, ce qui nuit à la standardisation.
- **Difficulté à gérer les interactions non hiérarchiques** dans les systèmes complexes.
- **Reconception fréquente en maintenance** : les fonctions sont plus volatiles que les données.

Cette approche s'avère particulièrement adaptée aux projets dont les processus métiers sont clairement établis et dont l'évolution est limitée, garantissant ainsi une modélisation efficace et peu coûteuse.

2.2. L'analyse systémique (2ème génération) : modéliser données et traitements

Principe :

Apparue dans les années 1980 avec des méthodes comme **Merise**, **Axial** ou **Information Engineering**, cette approche distingue :

- **L'aspect statique** : modélisation des **données** (modèle Entité-Relation de Chen) et de leurs liens sémantiques.
- **L'aspect dynamique** : modélisation des **traitements** (réseaux de Pétri ou formalismes dérivés), incluant l'état des données et les actions qui les modifient.

Apports récents :

- Spécification des **contraintes entre entités**, mécanismes d'**héritage**, et **normalisation** des données.

Points forts :

- **Cohérence des données** : bases de données non redondantes.
- **Respect des niveaux d'abstraction** : conceptuel, organisationnel, logique, physique, pour une décomposition progressive des besoins.

Points faibles :

- **Manque de cohérence entre modèles de données et de traitements** : absence de règles formelles pour contrôler cette cohérence.
- **Distinction des niveaux d'abstraction variable** selon les méthodes.
- **Critiques sur la lourdeur** : le niveau conceptuel est peu adapté aux maintenances et aux développements rapides.
- **Mélange des connaissances et du contrôle** : les règles de gestion et les contraintes d'intégrité sont modélisées dans une même logique algorithmique.

Dans les projets où la **cohérence des données** et la **traçabilité** constituent des exigences centrales, cette méthodologie offre un cadre rigoureux pour structurer les flux d'informations et assurer leur intégrité tout au long du cycle de vie du projet.

2.3. L'analyse objet (3ème génération) : modéliser objets et interactions

Principe :

Issue des langages de programmation des années 1970 et popularisée dans les années 1990, l'approche objet modélise le système comme un **ensemble d'objets indépendants et en interaction**, chaque objet combinant **données (attributs)** et **comportement (méthodes)**.

Principaux apports :

- **Réduction de la redondance** : à la fois pour les données (comme l'approche systémique) et pour les traitements (contrairement à l'approche cartésienne).
- **Meilleure modularité** : favorise la réutilisabilité des composants.
- **Représentation de la dynamique des objets** : facilite la modélisation des interactions complexes.

Méthodes emblématiques :

- **OOA** (Shlaer et Mellor), **OOD** (Booch), **OMT** (Rumbaugh), **UML** (Unified Modeling Language), fruit de la synthèse des méthodes objets.

Points forts :

- **Réduction des distorsions** entre le réel et le système informatique.
- **Modélisation facilitée** des objets complexes et de leur dynamique.
- **Encapsulation** : les mécanismes d'implémentation sont masqués derrière des interfaces claires.

Points faibles :

- **Tentation du « tout objet »** : risque de modéliser chaque concept comme un objet, même s'il ne nécessite pas de comportement complexe.
- **Positionnement bas dans le cycle de vie** : focus sur le **comment faire** (services, interactions) plutôt que sur le **quoi faire** (processus métier indépendants des technologies).

Pour les projets complexes exigeant une architecture **modulaire** et une forte **réutilisabilité** des composants, cette méthode permet de concevoir des systèmes évolutifs, maintenables et adaptables aux besoins changeants, tout en optimisant les coûts de développement et d'évolution.

2.4. Comparaison synthétique des trois approches

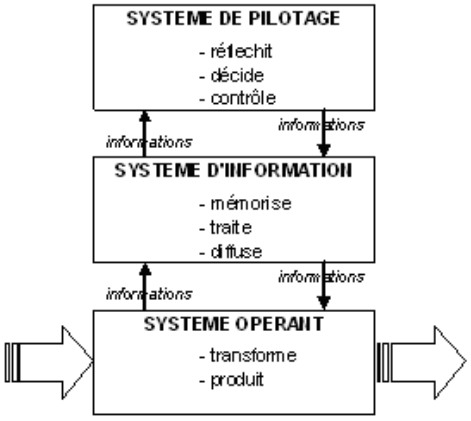
Critère	**Cartésienne**	**Systémique (Merise)**	**Objet**
Focus principal	Fonctions	Données et traitements	Objets et interactions
Cohérence	Faible (redondance des données)	Élevée (données non redondantes)	Élevée (données + traitements)
Flexibilité	Limitée	Moyenne	Élevée
Adaptabilité	Mauvaise (volatilité des fonctions)	Bonne	Excellente
Réutilisabilité	Faible	Moyenne	Élevée
Complexité des systèmes	Difficile à modéliser	Adaptée aux systèmes complexes	Adaptée aux systèmes complexes
Outils associés	SADT, Jackson	Merise, Axial	UML, OMT, OOSE, HOOD

3. Modélisation et stratégies : l'approche Merise et ses niveaux d'abstraction

3.1. L'approche systémique Merise : modéliser le système d'information

Merise propose une **modélisation systémique** du système d'information, le décomposant en trois sous-systèmes :

- **Le système opérant** : transforme les flux entrants et sortants.
- **Le système de pilotage** : maîtrise le système opérant.
- **Le système d'information** : représente le fonctionnement du système opérant à destination du système de pilotage.



Fonctions clés du système d'information :

- **Mémorisation** des informations nécessaires aux activités du système opérant et aux décisions du système de pilotage.
- **Exécution de traitements** sur ces informations.
- **Communication** des informations vers les acteurs et l'environnement.

Deux niveaux d'étude :

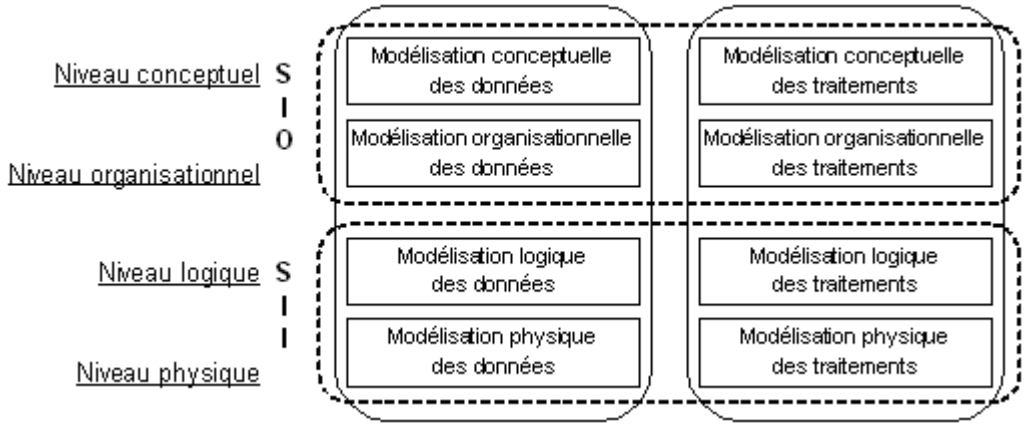
- **Système d'Information Organisationnel (SIO)** : centré sur l'organisation des activités, les ressources humaines et techniques, et la signification des informations.
- **Système d'Information Informatisé (SII)** : centré sur la construction de l'application informatique, les ressources logicielles/matérielles, et les bases de données.

3.2. Quatre niveaux d'abstraction pour quatre préoccupations distinctes

Merise distingue **quatre niveaux d'abstraction**, chacun reflétant un ensemble de préoccupations homogènes :

Niveau	Préoccupation	Description	Exemple
Conceptuel	« Que fait-on ? »	Représente les éléments descriptifs de l'activité du système, **indépendamment de toute organisation ou technologie**. Inclut les **règles de gestion**.	Modèle Entité-Relation (MER), diagrammes de flux.
Organisationnel	« Comment et avec quels moyens ? »	Décrit **qui fait quoi**, **où**, **quand** et **comment** (manuellement ou informatiquement).	Organigrammes, planning, répartition géographique.
Logique	« Comment et avec quels outils ? »	Représente comment le système est traité dans l'application informatique (bases de données, volumes).	Modèle logique des données, choix des SGBD.

Niveau	Préoccupation	Description	Exemple
Physique	« Comment ? »	Décrit les **choix techniques** et les ressources effectives (langages, architectures matérielles).	Code source, scripts de création de tables.



Séparation artificielle mais nécessaire :

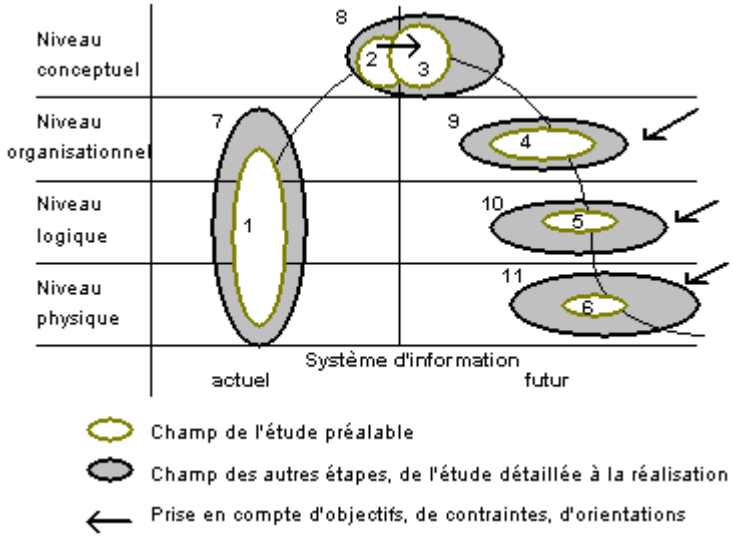
Les données et les traitements sont distingués pour regrouper des préoccupations homogènes, mais ils forment un tout indissociable. Cette séparation permet une **validation permanente** des modèles :

- Les données nécessaires aux traitements sont-elles bien modélisées ?
- Les traitements permettant de créer, modifier ou supprimer ces données sont-ils définis ?

3.3. Stratégie de passage de l'actuel au futur système d'information

Merise propose une **stratégie progressive** pour passer du système actuel au système cible, illustrée par la **« courbe du soleil »** :

1. **Comprendre le système actuel** (niveau physique et logique/organisationnel) :
    - Étudier les fichiers, documents, manuels, et réaliser des entretiens avec les acteurs.
2. **Modéliser le niveau conceptuel** du système actuel :
    - Extraire les règles de gestion et les processus métier **sans détailler** les modèles de données et de traitements.
3. **Définir les choix conceptuels** du futur système :
    - Élaborer les modèles conceptuels que le futur système devra respecter.
4. **Prendre en compte les choix organisationnels et techniques** :
    - Affiner les modèles avec les contraintes d'organisation et les outils technologiques (niveaux organisationnel, logique et physique).



Progression itérative :

La conception suit un **parcours itératif** à travers les modèles d'abstraction, en affinant progressivement la solution retenue. Chaque niveau est détaillé et généralisé, permettant une **traçabilité** entre les besoins métiers et la solution technique.

3.4. Modélisation : un outil au service de la qualité

La modélisation est l'outil principal de représentation du système d'information, construite **progressivement** au fil des étapes du projet. Merise propose **huit modèles** adaptés à chaque niveau d'abstraction, mais leur utilisation dépend de la **nature du projet** :

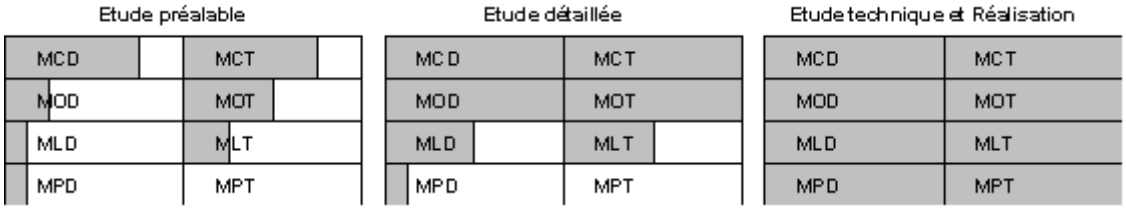
- **Domaine orienté « traitements »** : complexité au niveau des algorithmes (ex : relances, rapprochements de données).
- **Domaine orienté « données »** : complexité au niveau de la structuration des informations (ex : gestion de personnel, contrats).
- **Domaine orienté « communication »** : nombreux échanges de données et règles de gestion complexes (ex : workflows).

Précautions pour garantir la qualité :

- **Valider les concepts et principes d'organisation** au regard de la technologie envisagée (maquettage).
- **Ne pas laisser les contraintes techniques contraindre la conception** : les règles de gestion doivent primer.

Synthèse des stratégies Merise :

- **Étude préalable** : comprendre le système actuel et définir les axes pour le système cible.
- **Études détaillées** : réaliser l'application informatique, en affinant les modèles à chaque niveau.
- **Maquettage** : valider les modèles avec les utilisateurs (maquettes de bases de données, écrans, états).



3.5. Quand et pourquoi choisir Merise ?

Merise est particulièrement adaptée pour :

✅ Les projets où la **traçabilité** et la **cohérence** sont cruciales

✅ Les organisations ayant une **culture systémique** et une expérience des niveaux d'abstraction.

✅ Les projets nécessitant une **documentation complète** pour la maintenance et les évolutions futures.

Bibliographie / sources :

1. Fondements et enjeux de l'analyse informatique

- https://www.labri.fr/perso/johnen/Enseignement/UML.html
- http://b.phan95.free.fr/Reseau/DAGL/methode%20d%27analyse.pdf
- https://www.labri.fr/perso/guibert/DocumentsEnseignement/ACSI_v2.pdf

2. Les méthodes d'analyse : cartésienne, systémique et objet

- https://fr.wikipedia.org/wiki/Analyse_syst%C3%A9mique
- https://journals.openedition.org/methodos/4971
- https://fr.wikipedia.org/wiki/Cart%C3%A9sianisme
- https://www.coursaline.com/support-cours/Comprendre%20les%20m%C3%A9thodes%20d%27analyse%20et%20de%20conception.pdf
- http://sorcerer.fr.free.fr/info/proba/probatoire.htm

3. Modélisation et stratégies : l'approche Merise et ses niveaux d'abstraction

- https://fr.wikipedia.org/wiki/Merise_(informatique)
- http://tony3d3.free.fr/files/La-methode-Merise.pdf
- https://cours.pise.info/modelisation/merise.htm
- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/